# 스키마에 정의하는 7가지

**노드·관계 타입과 각각의 속성, 연결 패턴이 뼈대 — 제약과 허용 범위로 무엇을 지킬지 정함**

| 뼈대 ① ~ ⑤ | 정의할 것 · GraphSchema 칸 · 카드 예 | |
|---|---|---|
| ① 노드 타입 | node_types[].label | **Card, Benefit** |
| ② 노드 속성 | node_types[] .properties | **Card {name, annual_fee}** |
| ③ 관계 타입 | relationship_types[] .label | **HAS_BENEFIT** |
| ④ 관계 속성 | relationship_types[] .properties | **HAS_BENEFIT {valid_until}** |
| ⑤ 연결 패턴 | patterns (출발, 관계, 도착) | **(Card, HAS_BENEFIT, Benefit)** |
| **지킬 규칙 ⑥ ⑦** | | |
| ⑥ 제약 | constraints UNIQUENESS · EXISTENCE · KEY | **Card.name = KEY (필수 + 고유)** |
| ⑦ 허용 범위 | additional_* | **목록 밖은 지움 (목록을 주면 기본)** |




**7칸을 채운 Native GraphSchema — 실행 확인: name 없는 Card는 가지치기가 지움**

```
schema = GraphSchema.model_validate({
  "node_types": [                          # ① ②
    {"label": "Card", "properties": [
       {"name": "name", "type": "STRING"},
       {"name": "annual_fee", "type": "INTEGER"}]},
    {"label": "Benefit", "properties": [...]}],
  "relationship_types": [{"label": "HAS_BENEFIT",   # ③ ④
     "properties": [{"name": "valid_until", "type": "DATE"}]}],
  "patterns": [("Card", "HAS_BENEFIT", "Benefit")],   # ⑤
  "constraints": [{"type": "KEY", "node_type": "Card",  # ⑥
                   "property_names": ["name"]}],
  "additional_node_types": False})              # ⑦
```

**최소한 = ① · ③ · ⑤ + ⑥ 고유키, 속성은 질문에 쓰는 것만 · 타입마다 description을 적으면 LLM이 고르는 기준이 됨**
**최대 홉 수 · 결과 상한 · 상충 관계는 검색 규칙이라 스키마에 넣지 않음 · 값 형식(type)은 아직 검사 안 함 → 저장 전 따로 변환**

무단전재 및 배포 금지